

Binary Activation

Identity-Bound FFN Confinement for Shared Neural Networks

Ryan McCormick¹

¹Independent Researcher

August 2026

Abstract

Public-key infrastructure made identity administrable by binding keys to names across organizations. Shared model artifacts need an analogous addressable structure for capability: a signed grant must identify not only who may execute, but which internal support it authorizes. Binary activation supplies that structural basis at declared neural boundaries.

Shared neural networks need a mechanism that determines not only who may call a model, but which learned coordinates a credential may activate. We call this mechanism *binary activation*. An authenticated credential resolves to a binary mask over a feed-forward network (FFN) activation. The model computes $\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r$: coordinates assigned to regime r are preserved, and every excluded coordinate is exactly zero. The same multiplication governs backpropagation. When the gate is placed on the expanded FFN activation before the down projection, loss gradients and optimizer state can be confined to the aligned columns of the first projection, hidden bias entries, and rows of the second projection.

We formalize the local gate, gradient, and aligned-weight statements in Lean 4 and report seven complementary experiment families. A five-seed formative study finds post-encoder co-training costs of 0.14–0.38 percentage points across $R = 8$ –128; no paired test survives Bonferroni correction. In the stricter frozen-backbone DistilBERT study, every intermediate FFN is gated and only authorized FFN slices plus a private classifier are trainable. Across three seeds, owning accuracy falls from 91.28% at $R = 1$ to 86.21% at $R = 16$, while frozen shared weights, off-partition parameters and optimizer moments, and inactive classifiers remain bit-exact unchanged. Private residual adapters recover 90.47% owning accuracy at $R = 4$ without modifying inactive lanes. A capacity-preserving synthetic classifier with $d = 10R$ obtains 256/256 owning predictions at $R = 128$ and zero hits in 32,512 wrong-key probes; authorized support extraction has maximum logit difference zero. It is a memorization and scaling result, not held-out utility evidence. Exact extraction, full-capacity inference controls, and deliberately broken placement controls distinguish zero-cost authorization and reparameterization from the measured cost of dividing FFN capacity. The claim is intentionally local: binary activation supplies exact coordinate exclusion at a correctly placed FFN gate, not complete-model privacy through shared attention, caches, logs, or undeclared bypasses.

1 Introduction

Public-key infrastructure answers whose key is being presented and who vouches for it. As trained models are licensed, co-located, and embedded in systems outside their creator’s custody, an analogous question arises: which capability inside this artifact may this authenticated principal execute? Encryption and endpoint admission alone do not provide an internal object to which a scoped, signed grant can refer. Binary activation supplies such an addressable object at a declared model or FFN boundary.

Multi-tenant inference commonly treats authorization as a routing decision: after a caller is admitted, a serving layer chooses a model, adapter, or data source. That is useful, but it

does not define a structural boundary inside a shared learned representation. Binary activation connects an authenticated principal to a computation-level decision. A credential is verified by a trusted authority, resolved to a regime, and converted into a binary support mask. The model can then use an FFN coordinate if and only if the resolved mask activates it.

The core operation is elementary:

$$\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r, \quad \mathbf{g}_r \in \{0, 1\}^d. \quad (1)$$

Its value comes from where it is placed and how it is controlled. At the gate, $0 \cdot x = 0$ eliminates an excluded coordinate for every input value, and $1 \cdot x = x$ preserves an active coordinate. By the chain rule, the same mask zeros the corresponding loss gradient. With an aligned optimizer, one credential can therefore read from and write to only its declared FFN support.

This paper contributes:

- a capability-PKI formulation in which signed, identity-bound grants address declared activation supports inside a model artifact;
- a deterministic keyed construction of disjoint and exhaustive regime masks;
- a precise FFN gate placement with forward, gradient, and aligned-state boundaries;
- Lean 4 formalization of the central algebraic statements;
- formative and strict DistilBERT classification studies measuring utility from $R = 1$ through $R = 128$ at two declared gate surfaces;
- a capacity-preserving classifier construction that holds each regime’s allocated width fixed while total width grows as $d = qR$;
- private-capacity lanes and exact inactive-state checks;
- exact extraction and full-capacity inference controls; and
- deliberately broken placement and bypass controls.

The central security statement is structural but narrow. It applies at the declared FFN activation and aligned trainable state. Identity verification, credential custody, mask dispatch, optimizer behavior, gate placement, and closure of bypass paths remain part of the trusted implementation boundary.

2 Classifier and Modular Use Cases

2.1 Why the DistilBERT experiments worked

DistilBERT is a Transformer. The relevant boundary is therefore not “Transformer” versus “non-Transformer,” but which part of the computation the gate governs. In the formative classifier study, DistilBERT supplies a shared Transformer encoder and the gate acts on the final CLS representation. In the strict study, a gate acts inside each Transformer block’s FFN while attention, normalization, embeddings, and residual paths are frozen and shared. Neither construction claims to partition attention.

The formative post-encoder result worked because the complete encoder was co-trained with the masked CLS representation. Optimization could therefore place discriminative features onto the supports that each regime would retain. That experiment measures utility under a learned support constraint; because all regimes update the shared encoder, it is not evidence of separated tenant state inside DistilBERT.

The strict result works for a different reason. A Transformer FFN already constructs an explicit expanded tensor after an element-wise nonlinearity and before a linear down projection. Its coordinates, aligned projection slices, and optimizer entries are identifiable constituent pieces on which a binary support can act. Freezing attention, embeddings, normalization, and residual parameters leaves a shared feature-producing backbone while the governed FFN slices and private classifier adapt. The resulting capacity curve is therefore a valid Transformer-classification result, not an attention-partition claim.

2.2 Classifier uses and capacity policies

This still covers a broad and commercially useful classifier surface. Examples include identity-scoped intent and document routing, fraud and abuse scoring, policy and entitlement decisions, risk tiers, content moderation, and classification heads attached to vision or audio encoders. A regime need not mean one entire tenant: the authority may bind a support to a tenant plus a sub-identity, role, licensed capability, or data purpose, and may authorize a declared union when one principal is entitled to several capabilities.

Three patterns avoid treating a model as one indivisible dense object:

- an atomic whole-model gate authorizes an unchanged classifier before execution and therefore does not alter its numerical computation;
- a shared frozen encoder feeds a gated classifier, FFN slice, private adapter, or private expert; and
- a block-sparse classifier or expert bank is sized with a fixed width q per regime, so total governed width is $d = qR$ rather than a fixed d divided into progressively smaller pieces.

The third pattern preserves allocated per-regime capacity by spending storage linearly in R . It is natural for branched classifiers, expert banks, adapters, and other modular surfaces. Binary activation supplies identity-bound selection and exact zeroing at the declared gate; sparse dispatch or authorized extraction may avoid evaluating inactive blocks. It does not make additional private capacity free.

2.3 Why attention is a different space

Binary Activation requires more than a tensor on which multiplication is defined. The protected computation must expose stable constituent pieces: an indexed support before cross-support mixing, aligned parameters and optimizer state, and no ungated path around the support boundary. Standard self-attention does not expose those pieces as fixed tenant coordinates. Queries and keys form input-dependent pairwise dot products. Those products encode both vector norms and angular proximity; when norms are controlled, the angular term corresponds to cosine adjacency. Softmax then couples every candidate through a shared denominator, the output projection recombines heads, and normalization and residual additions mix or bypass the result.

Masking an arbitrary attention coordinate still produces an exact zero at that coordinate, but it does not construct an independent tenant constituent from a distributed relational representation. A fixed mask can change token affinities and the softmax distribution globally, which explains why post-hoc head ablation damages utility. To make attention partitionable, the architecture would have to construct complete lanes from the outset—including Q/K/V, output projection, normalization, FFN, residual stream, and cache state—and gate every route before recombination. That is possible as a research design, but it is a different and substantially higher-risk construction than the DistilBERT classifier experiments reported here.

3 Binary Activation Model

3.1 Credential-to-support resolution

Let c be an authenticated credential associated with a user, agent, or tenant. A trusted authority resolves c to a regime identifier r and the corresponding mask $\mathbf{g}_r$. Callers do not supply masks or choose the underlying coordinate assignment. The model input is therefore the pair $(\mathbf{x}, c)$, not $\mathbf{x}$ alone:

$$c \mapsto r \mapsto \mathbf{g}_r, \quad M(\mathbf{x}, c) = M(\mathbf{x}; \mathbf{g}_r). \quad (2)$$

Authentication and algebra perform different jobs. Authentication determines which support is authorized; multiplication enforces that support at the declared tensor. Knowledge of mask

indices is not itself a credential when the runtime refuses caller-supplied assignments.

The resolved regime then becomes an authenticated execution identity for each downstream gate that verifies the same bound grant, model, operation, and policy scope. The resulting chain is external authentication, boundary authorization, and downstream regime authentication. A bare regime integer or activation tensor does not authenticate itself. Identity assurance terminates at the verifier-configured PKI root and the operator’s certificate and key-custody controls; PKCS#12 credential packaging alone does not prove hardware residency.

3.2 Disjoint keyed assignment

Given hidden dimension $d \geq R$, a gate partition is a family $\{\mathbf{g}_0, \dots, \mathbf{g}_{R-1}\} \subset \{0, 1\}^d$ satisfying

$$\mathbf{g}_r \odot \mathbf{g}_s = \mathbf{0} \quad (r \neq s), \quad \sum_{r=0}^{R-1} \mathbf{g}_r = \mathbf{1}. \quad (3)$$

Every coordinate belongs to exactly one regime. Equal-width partitions require $R \mid d$, equivalently $d = qR$ for an integer $q \geq 1$; otherwise block sizes may differ by at most one.

The reference construction uses HMAC-SHA256 as a pseudorandom function to drive Fisher–Yates over the coordinate indices. The encoded context includes a model digest, model version, R , d , the shuffle position, and a rejection-sampling counter. Accepted 256-bit words are reduced modulo the current shuffle range only after rejecting the incomplete residue tail. The resulting permutation is partitioned into R blocks and converted to binary masks.

The keyed shuffle makes assignment deterministic for the same key and context and unpredictable under the standard PRF assumption. Disjointness and exhaustiveness come from partitioning the permutation; the local zero at the gate comes from the binary mask, not from secrecy of the permutation.

3.3 Computational overhead

The runtime gate is one element-wise multiply over d coordinates per gated tensor: d scalar multiplications with no accumulation, rather than an extra matrix product. Writing the operation as $\text{diag}(\mathbf{g}_r)\mathbf{h}$ and evaluating that dense matrix would cost $\Theta(d^2)$ operations and would be an implementation error. The intended broadcast multiply is $\Theta(d)$ against the $\Theta(d d_{\text{in}})$ and $\Theta(d d_{\text{out}})$ projection costs around it, and it may be fused with a surrounding kernel.

For inference-output equivalence, a constant mask may instead be folded into the aligned rows of the down projection. That packaging transformation removes the per-request multiply, but it does not instantiate the declared activation-level zero or the training-state confinement boundary; it is an extracted or reparameterized artifact. A bit-packed mask requires d bits (96 bytes at $d = 768$), while an expanded Boolean or floating-point tensor uses more implementation-dependent storage. No dedicated gate-overhead microbenchmark is included, so this paper makes no numerical latency claim.

4 FFN Placement and Confinement

4.1 Intermediate-FFN activation

For row-vector activations, write a two-projection FFN as

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1), \quad (4)$$

$$\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r, \quad (5)$$

$$\hat{\mathbf{y}} = \tilde{\mathbf{h}}\mathbf{W}_2 + \mathbf{b}_2. \quad (6)$$

The gate lies after the element-wise nonlinearity and immediately before the down/output projection. An excluded hidden coordinate contributes zero to every output coordinate regardless of its pre-gate value.

The location is essential. If normalization or another mixing operation moves information between coordinates before the mask is applied, a later zero cannot undo signal already transferred into active coordinates. Likewise, an ungated residual, adapter, cache, or other parallel branch lies outside the claim.

4.2 Forward and gradient properties

For coordinate j :

$$g_{r,j} = 0 \implies (\mathbf{h} \odot \mathbf{g}_r)_j = 0, \quad (7)$$

$$g_{r,j} = 1 \implies (\mathbf{h} \odot \mathbf{g}_r)_j = h_j. \quad (8)$$

Backpropagation gives

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}} \odot \mathbf{g}_r, \quad (9)$$

so $g_{r,j} = 0$ implies $\partial \mathcal{L} / \partial h_j = 0$ for every loss, input, and batch size.

In the notation of Equation (6), the aligned loss gradients are therefore zero for excluded columns $\mathbf{W}_1[:, j]$, hidden-bias entry $b_{1,j}$, and rows $\mathbf{W}_2[j, :]$. The output bias $\mathbf{b}_2$ is not indexed by the hidden partition and is shared unless separately frozen, partitioned, or gated.

4.3 Parameter state and optimizer state

Zero loss gradient alone does not stop every optimizer from moving a parameter. Decoupled weight decay, momentum accumulated under another regime, or an unmasked post-processing step can modify an inactive coordinate. Exact parameter-state confinement therefore requires one of the following equivalent implementation patterns:

- optimizer moments stored and updated only for the active support;
- inactive weights and moments restored after each step; or
- an optimizer whose complete update, including weight decay, is masked by the same regime support.

Under that condition, regime- r training updates only the aligned FFN slices and any explicitly private state such as a regime-selected classifier. Shared attention, embeddings, normalization, output heads, and runtime state do not inherit this guarantee.

4.4 Per-sample co-training

Binary activation supports mixed-regime minibatches. For examples $\{(\mathbf{x}_i, y_i, r_i)\}_{i=1}^B$, each sample uses its own mask:

$$\tilde{\mathbf{h}}_i = \mathbf{h}_i \odot \mathbf{g}_{r_i}. \quad (10)$$

The resulting gradients can be accumulated in one optimizer step while each sample remains confined to its support. The supports may be assigned to different tenants or to multiple capabilities held by one principal. A runtime that intentionally authorizes a union removes the boundary among the included supports; coordinates outside the union remain excluded.

5 Formal Verification

The accompanying pinned Lean 4 project builds 21 modules and 317 active theorem or lemma declarations, with no active `sorry` or `admit` declarations. A checked-in `#print axioms` audit shows that thirteen selected gate, placement, cardinality, and output statements use only

Lean/Mathlib’s standard logical axioms. Four project-specific `axiom` declarations are present in separate conditional or historical theorem chains; none reaches the headline statements used here.

The historical recovery-cost chain now takes an explicit, unvalidated `FullEnumerationAssumption` as a caller premise. The size of a support space does not establish attack cost or a pseudorandom-function reduction. Likewise, softmax normalization establishes a valid probability distribution, not answer correctness, confidence, or concealment of an incorrect regime.

The selected results are:

- `gradient_isolation`: an inactive gate coordinate has zero activation gradient;
- `weight_update_confined`: first-projection updates are confined to active hidden columns;
- `w2_update_confined`: second-projection updates are confined to active hidden rows;
- `masks_orthogonal`: distinct regime masks have disjoint support;
- `residual_block_full_confinement`: the aligned FFN residual branch remains confined for arbitrary upstream gradient;
- `non_preserving_breaks_w1_confinement`: an intervening non-support-preserving operation such as LayerNorm invalidates the first projection confinement statement; and
- `compressed_eq_gated`: selecting active support yields the same idealized linear result as full gated execution.

These are local algebraic theorems. They do not prove the correctness of an identity provider, key service, tensor-axis mapping, optimizer implementation, or the absence of bypass paths.

6 Experimental Evaluation

6.1 Evidence map

The experiments answer different questions and are not pooled into one claim. The first study measures whether gate-aware optimization can use very narrow supports. The wide-classifier study instead holds each regime’s allocated width fixed while total width grows with R . The strict study then moves the gate inside every FFN and audits parameter and optimizer state. Private lanes test the effect of restoring per-regime capacity. Extraction and full-capacity controls test exact inference preservation when the authorized computation is not capacity constrained. Finally, negative controls show that the harness detects invalid placement and an ungated residual bypass.

Table 1: Completed experiment families included in this evaluation.

Experiment and repetition	Headline result
Post-encoder co-training: 5 paired seeds; 7 values of R	Observed gap 0.143–0.376 percentage points
Capacity-preserving classifier: 1 deterministic $R = 128$ run	256/256 owning; 0/32,512 wrong-key hits; selected/dense difference 0
Strict intermediate FFN: 3 seeds; $R = 1, 2, 4, 8, 16$	Bit-exact inactive state; measured capacity curve
Private residual lanes: 3 seeds; 2 designs at $R = 4$	90.47% adapter accuracy; inactive-lane delta 0
Authorized extraction: 10 seeds; 160 comparisons	80/80 fp32 bit-identical; 80/80 fp16 within tolerance
Full-capacity controls: 1 trained model plus local authorization probe	Zero measured inference difference when capacity is retained
Placement controls: 1 deterministic local protocol run	Correct gate 0; invalid LayerNorm and residual paths nonzero

All reported values are backed by checked-in runners and machine-readable artifacts. The

formative statistics are in `experiments/results/series1_results_20260803_202919.json`; strict FFN and private-lane runs are in the corresponding `dense_ffn_cotenancy_*.json` and `private_transformer_lanes_*.json` files; extraction is in `local_exact_extraction_20260803_172654.json`; the full-basis sweep is in `full_run_r128_log.txt` and `r1024_test_20260622_103122.json`; and the whole-model, pre-MLP, and broken placement checks are in `archive/smoke/transformer_cotenancy_local_20260812_100546.json`. The capacity-preserving classifier artifact is `capacity_preserving_wide_20260820_225331.json`. The repository README maps each artifact to its public runner.

6.2 Formative co-training at narrow supports

The formative study applies a regime mask to DistilBERT’s final 768-dimensional CLS vector after the encoder. It therefore tests support-constrained task learning, not intermediate-FFN parameter confinement. At each R , one gated model trains all regimes while the paired separate-model control trains four sampled regimes. Each statistic compares those same four regimes across seeds 42, 123, 256, 456, and 789.

Table 2: Five-seed paired post-encoder co-training study. Gap is separate minus gated accuracy. Intervals use Student’s t distribution with four degrees of freedom.

R	Dims/regime	Gap (pp)	95% CI (pp)	Paired p
8	96	0.348	[−0.229, 0.925]	0.1692
16	48	0.268	[−0.301, 0.837]	0.2616
24	32	0.208	[−0.324, 0.740]	0.3392
32	24	0.267	[−0.345, 0.879]	0.2920
64	12	0.143	[−0.041, 0.326]	0.0967
96	8	0.376	[0.045, 0.708]	0.0345
128	6	0.272	[−0.208, 0.752]	0.1902

The observed costs are small despite supports as narrow as six coordinates. Only $R = 96$ crosses an unadjusted 0.05 threshold; none crosses the Bonferroni family-wise threshold $0.05/7 \approx 0.0071$. Failure to reject zero is not proof of equality, and exact equality is not expected between independently optimized model inscriptions. The defensible result is the measured effect size: in this formative study, support-constrained co-training remains within 0.38 percentage points of the sampled separate-model controls at every tested ratio.

Replay. Use the legacy post-encoder surface in

`experiments/reproduce_distilbert_classification.py` with the reported five-seed rows retained in `experiments/results/series1_results_20260803_202919.json`.

6.3 Holding per-regime classifier capacity fixed

The preceding sweep fixes the 768-dimensional post-encoder representation, so each support narrows as R grows. That is not the only sizing policy. If a task needs q hidden coordinates per regime, the architecture may instead choose $d = qR$. Adding a regime then adds a block without reducing any incumbent regime’s allocated width.

We ported and reran the formative wide-classifier protocol with $R = 128$, $q = 10$, and therefore $d = 1,280$. The two-layer MLP has 256 one-hot synthetic observations and 256 disjoint answers, two per regime. It is co-trained for 2,000 full-batch epochs using a post-activation binary gate. The sparse selected-support implementation is also compared with explicit dense-mask execution.

The owning key produces the expected answer for 256/256 observations. Across all 32,512 pairs of a foreign observation and wrong regime key, the expected foreign answer is never produced. Activating all 128 regimes simultaneously produces 249/256 correct answers, and selected-support

versus dense-mask logits have maximum absolute difference 0. The full runner and JSON artifact are checked in.

This result is deliberately narrow. It is a deterministic synthetic memorization and scalability test with no held-out set, and the zero wrong-key hit count is an empirical negative control rather than a privacy theorem. It does show why “ R regimes” need not imply “ $1/R$ of a fixed model” for modular classifiers: each regime retains the same ten-coordinate allocation because the governed layer grows with R . The cost is linear parameter storage, not lost per-regime width.

Replay. The self-contained runner is `experiments/capacity_preserving_wide_classifier.py`; the reported run is `experiments/results/capacity_preserving_wide_20260820_225331.json`.

6.4 Deployment service-consolidation consequence

An $R = 8$ DistilBERT SST-2 deployment benchmark compares eight complete model services with three single-backbone layouts. Eight services require 1.071 GB of checkpoint tensors and 1.103 GB of resident CUDA parameter and buffer state. A frozen shared backbone with eight zero-initialized private adapter slots retains the original 91.06% accuracy using 151.6 MB and 156.2 MB, respectively. This is a deployment control, not evidence that untrained adapters add utility.

Post-hoc one-eighth FFN slicing reduces accuracy to 67.66%. Physically extracting the authorized slices preserves the sliced logits within the declared fp16 tolerance and raises measured throughput from 1,585 to 2,872 samples/s. Every scored request passes through execution authority and malformed authority makes zero model calls. Thus the measurement supports consolidating a frozen backbone plus private modules, while directly rejecting the stronger claim that naive narrowing of a trained dense Transformer preserves utility. Timing is one serial stream on one NVIDIA T4 and is not a concurrency result.

Replay. Run `experiments/modal_distilbert_service_consolidation.py`; the full artifact is listed below.

`experiments/results/distilbert_service_consolidation_20260821T135530_707107Z.json`.

6.5 Strict intermediate-FFN cotenancy

6.5.1 Protocol

We evaluate DistilBERT-base [Sanh et al., 2019] on AG News [Zhang et al., 2015]. Attention, embeddings, normalization, residual parameters, and every other non-governed shared weight are frozen. A binary gate is placed on the 3,072-dimensional expanded activation immediately before every FFN down projection. For regime r , training may update only the active first-projection hidden rows, corresponding hidden bias entries, matching second-projection columns in PyTorch storage convention, and a private regime-selected classifier. A regime-scoped Adam implementation updates moments only on authorized entries.

The model is trained on 8,000 examples for two epochs and evaluated on 2,000 held-out examples at seeds 42, 123, and 256. Label permutations supply regime-specific functional identities. The $R = 1$ arm is a matched full-FFN capacity reference under the same frozen-backbone protocol.

6.5.2 Interpretation

The unauthorized-delta column is the maximum over frozen shared parameters, off-partition FFN parameters, off-partition first and second optimizer moments, and inactive private classifiers. Every category is bit-exact zero in every run. The utility cost is empirical and architecture-specific: halving each FFN retains 90.15% accuracy, 1.13 percentage points below the full-capacity reference, while the 192-coordinate slice at $R = 16$ incurs a 5.07-point drop.

Table 3: Strict intermediate-FFN cotenancy. Accuracy is mean $\pm$ sample standard deviation over three seeds.

R	Dims/tenant/layer	Owning accuracy	Drop (pp)	Max unauthorized delta
1	3,072	91.28% $\pm$ 0.33%	0.00	0
2	1,536	90.15% $\pm$ 0.39%	1.13	0
4	768	88.93% $\pm$ 0.30%	2.36	0
8	384	87.47% $\pm$ 0.04%	3.82	0
16	192	86.21% $\pm$ 0.08%	5.07	0

This experiment does not prove parity and does not need to. Independently optimized model inscriptions have ordinary variance. The experiment measures a capacity curve under a strict state boundary. The exact result is inactive-state preservation; the measured result is owning-task utility.

A current-library seed-42 rerun at $R = 8$ independently records 87.756% owning accuracy and exact zero for the same frozen, off-partition, optimizer-moment, and inactive-classifier deltas. Scored evaluation first passes through the source-visible research preflight’s model, dimension, and regime checks; wrong model, dimension, regime, and empty-authority probes make zero model calls. This one seed corroborates the boundary but does not replace the three-seed utility estimate above.

Replay. Run `experiments/modal_dense_ffn_cotenancy.py`; the three-seed artifacts are the `experiments/results/dense_ffn_cotenancy_*.json` files and their aggregate is `experiment s/results/transformer_cotenancy_summary.json`.

6.6 Restoring private capacity

The strict disjoint-slice construction gives each regime d_{ff}/R trainable coordinates. A complementary $R = 4$ experiment asks what happens when each regime instead receives a complete private module on the same frozen shared DistilBERT backbone. The adapter design adds a 64-dimensional residual adapter after every block; the expert design adds a complete 3,072-dimensional private FFN after every block. The regime-selected classifier is private in both designs, while attention remains frozen and shared.

Table 4: Complete private lanes on frozen DistilBERT at $R = 4$. Accuracy is mean $\pm$ sample standard deviation over seeds 42, 123, and 256.

Design	Params/regime	Owning acc.	Wrong-key acc.	Max delta
64-dim residual adapters	0.59M	90.47% $\pm$ 0.18%	3.18% $\pm$ 0.06%	0
3,072-dim private experts	28.33M	90.09% $\pm$ 0.14%	3.30% $\pm$ 0.05%	0

The adapter is the best measured capacity tradeoff: it recovers most of the full-FFN utility with much less private state than the expert. The max-delta column covers the frozen shared backbone and every inactive lane. Wrong-key accuracy is a functional negative control induced by the regime-specific label permutations, not the source of the structural claim. This design restores private capacity rather than dividing one trainable private module by R ; its cost is per-regime storage.

A current-library seed-42 rerun records 90.66% adapter and 90.15% expert owning accuracy, with 3.11% and 3.28% wrong-key accuracy. Shared-backbone and inactive-lane deltas remain exactly zero, and malformed execution authority makes zero model calls. These one-seed values corroborate, rather than replace, the historical three-seed estimates.

Replay. Run `experiments/modal_private_transformer_lanes.py`; the reported adapter and expert runs are `experiments/results/private_transformer_lanes_*.json`.

7 Exact Authorized Extraction

Let $A_r = \text{supp}(\mathbf{g}_r)$. For a projection $\mathbf{W} \in \mathbb{R}^{4 \times 768}$, input $\mathbf{X} \in \mathbb{R}^{256 \times 768}$, and bias $\mathbf{b}$, compare

$$(\mathbf{X} \odot \mathbf{g}_r) \mathbf{W}^\top + \mathbf{b} \quad \text{and} \quad \mathbf{X}[:, A_r] \mathbf{W}[:, A_r]^\top + \mathbf{b}. \quad (11)$$

The extracted expression removes every inactive product rather than computing it and multiplying by zero.

Across ten seeds, four individual regimes, authorized unions of one through four regimes, and both 32-bit and 16-bit arithmetic, all 160 comparisons satisfy their declared numerical criteria. In the canonical 768-dimensional run, all 80 32-bit comparisons are bit-identical. This is an observed kernel-specific result, not a general floating-point theorem: smaller checks can differ by a few fp32 units in the last place because the two expressions use different reduction shapes. Of the 80 16-bit comparisons, 60 are bit-identical and the maximum absolute difference is 0.015625. Every comparison is within a conservative forward-error bound computed from the input, weights, support size, accumulator precision, and bias.

Extraction is both a consequence and a practical use of binary activation. A runtime may enforce the full mask dynamically, or an authority may export only the authorized support as a reduced dense artifact. The extracted artifact does not require the runtime multiplication because excluded coordinates are absent.

Replay. Run `experiments/local_exact_extraction.py`; the reported artifact is `experiment_s/results/local_exact_extraction_20260803_172654.json`.

8 Full-Capacity, Zero-Degradation Controls

The phrase “zero degradation” is exact only when the authorized computation retains all capacity it needs. It should not be inferred from a non-significant difference between independently trained models. Three completed controls make the distinction concrete.

Table 5: Inference-preservation controls. These retain the complete authorized computation; they do not claim that dividing an FFN into narrower supports is cost-free.

Control	Result and interpretation
Atomic whole-model activation	Maximum authorized output difference 0 after one unchanged model call; five wrong scopes make zero calls
Authorized extraction	Same authorized linear computation with inactive products removed: fp32 80/80 bit-identical; fp16 80/80 within tolerance
Keyed full-basis placement	Same trained function in a different coordinate basis: maximum accuracy gap 0.00 pp at $R = 4$ –128 and eight sampled placements at $R = 1,024$

The atomic authorization probe binds tenant, sub-id, model digest, and operation before execution. Its owning path makes one call and is numerically identical to direct invocation. Random, wrong-tenant, wrong-sub-id, wrong-model, and wrong-operation credentials make zero model calls. This is Binary Activation at the complete-model boundary: it preserves model quality because it does not alter the model.

The extraction result is likewise an implementation-equivalence result. It removes only products that the authorized binary mask already forces to zero. It says that a runtime may materialize the active support without changing the authorized linear result; it does not compare a narrow support with a wider model.

Finally, a full-capacity DistilBERT control conjugates every residual-facing parameter consistently into a keyed permutation basis. The base model and all tested placements score 90.25% on the same 7,600-example test set, with exactly zero prediction-level accuracy gap for all placements at $R = 4$ through $R = 128$ and eight sampled placements at $R = 1,024$. No dimension is removed

in this control, so the result is consistent with exact function preservation. Conjugation alone is not Binary Activation: it creates neither distinct tenant knowledge nor a zero on non-owning coordinates. It is included only to show that keyed placement itself has zero inference cost when the full basis remains available. Tenant confinement still requires a correctly placed gate and the capacity tradeoffs measured above.

Replay. Atomic whole-model authorization is exercised by `experiments/local_transformer_cotenancy_suite.py`; extraction by `experiments/local_exact_extraction.py`; and the keyed full-basis control by `experiments/orthogonal_superposition_sweep.py`. Retained outputs are listed in `experiments/results/README.md`.

9 Placement and Bypass Negative Controls

A deterministic local protocol run tests both the valid gate and two invalid constructions. At a correctly placed disjoint gate, the maximum cross-mask value is exactly zero. Inserting LayerNorm between the source assignment and the target gate produces a nonzero target coordinate of 0.12598. Adding an ungated residual route carries the excluded source with maximum magnitude 3.0. The controls therefore fail in the direction predicted by the threat model.

A separate pre-MLP probe gates a 32-dimensional FFN input before a 32-to-64 first projection at $R = 4$. The full gated and physically extracted projections have maximum difference 0; inactive input gradients and aligned inactive first-projection-row gradients are also exactly zero. This is evidence for the declared pre-MLP surface only, not for hidden/down-projection partitioning.

These controls matter because an always-zero test could otherwise reflect a dead harness. The observed nonzero failures show that the implementation detects the normalization and bypass conditions that invalidate a broader isolation claim.

Replay. Run `experiments/local_transformer_cotenancy_suite.py`. The retained protocol artifact is `experiments/results/archive/smoke/transformer_cotenancy_local_20260812_100546.json`.

10 Threat Model and Limits

The in-scope adversary is a caller that may submit chosen inputs, possess its own credential, alter an asserted regime identifier, or guess coordinate assignments. The trusted runtime verifies the credential, resolves the regime, obtains the authority-selected mask, and applies the gate at the declared FFN surface. It does not accept a caller-selected mask.

The following remain trusted or separately controlled:

- credential issuance, verification, and revocation;
- assignment-key custody and credential-to-mask dispatch;
- model and tensor-axis integrity;
- gate placement and closure of residual or other bypass routes;
- optimizer state and weight-decay behavior; and
- caches, logs, backups, retrieval stores, adapters, and other serving state outside the declared FFN parameter surface.

A wrong credential cannot activate the owning regime’s coordinates under correct dispatch because disjointness supplies an exact zero at that gate. That does not imply that the complete output is zero: the caller’s authorized coordinates remain active. Nor does it prove statistical independence of shared representations. Attention and other shared components may compute common features before or after the gated FFN; their parameters are not represented as private by this theorem.

Improper gate placement can nullify the intended isolation. If a protected signal is mixed into active coordinates before masking, or if an ungated branch carries it around the gate, multiplication still zeros the immediate tensor but does not isolate the complete computation.

A conforming implementation must state exactly which activation, parameter slices, optimizer state, and runtime state it claims to confine.

11 Related Work

Task-specific masking methods such as PackNet [Mallya and Lazebnik, 2018], Piggyback [Mallya et al., 2018], and Supermasks in Superposition [Wortsman et al., 2020] demonstrate that subnetworks can share a parameterized model. Binary activation differs in joining a credential-derived disjoint assignment to both inference and training-time support. LoRA [Hu et al., 2022] provides efficient private trainable state, but does not by itself supply an identity-bound binary activation boundary.

Formal verification of neural systems commonly models robustness or system properties at higher levels [Seshia et al., 2018]. Here the principal formal results are deliberately small: exact consequences of a binary Hadamard gate at a declared FFN boundary.

12 Conclusion

Binary activation connects identity to neural computation. A verified credential selects a disjoint FFN support; the Hadamard gate determines which coordinates can contribute to inference and where loss gradients can flow. At the correctly placed intermediate FFN activation, the local exclusion is exact. With aligned optimizer state, the corresponding parameter updates are exact as well.

The experiment set separates effects that a single accuracy table cannot. Formative co-training uses supports as narrow as six coordinates with observed costs below 0.38 percentage points. The wide-classifier experiment instead holds per-regime width fixed while scaling $d = qR$, and recovers 256/256 owning predictions at $R = 128$ on its synthetic memorization task. Strict intermediate-FFN cotenancy keeps inactive state bit-exact unchanged while utility declines as each regime receives less FFN capacity. Complete private lanes recover utility by restoring per-regime capacity at a storage cost. Exact extraction and full-capacity controls then show where zero inference degradation is the right statement: when authorization or reparameterization leaves the complete authorized computation available. Deliberately broken placement controls confirm that normalization and ungated residual paths invalidate broader claims.

These results do not require complete Transformer partitioning, generative validation, or a statistical assertion of zero leakage. They require a correctly placed binary gate, aligned optimizer state, and a trusted credential-to-mask runtime. Capacity-preserving controls are reported as such, not as evidence that every divided FFN partition is free.

In capability-infrastructure terms, the contribution is an identity-addressable internal support to which a scoped grant can refer; the runtime and lockbox architecture that issues and redeems those grants is treated separately.

Code, artifacts, and Lean proofs are available at <https://github.com/sekosai/schemen-gate/tree/main/research/cdp>.

References

- E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. <https://arxiv.org/abs/2106.09685>
- A. Mallya and S. Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018. <https://arxiv.org/abs/1711.05769>
- A. Mallya, D. Davis, and S. Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. <https://arxiv.org/abs/1801.06519>

- V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019. <https://arxiv.org/abs/1910.01108>
- S. A. Seshia, A. Desai, T. Dreossi, et al. Formal specification for deep neural networks. In *ATVA*, 2018. <https://arxiv.org/abs/1710.01688>
- M. Wortsman, V. Ramanujan, R. Liu, et al. Supermasks in superposition. In *NeurIPS*, 2020. <https://arxiv.org/abs/2006.14769>
- X. Zhang, J. Zhao, and Y. LeCun. Character-level convolutional networks for text classification. In *NeurIPS*, 2015. <https://arxiv.org/abs/1509.01626>